

## МЕТОДИЧЕСКИЕ УКАЗАНИЯ

по дисциплине Б1.О.18 «Основы проектной деятельности»

для выполнения практических занятий

Курс: 1 Семестр: 2

Форма работы: командная (2–3 человека в команде)

---

*Общие сведения о курсе*

Организационная структура

Дисциплина рассчитана на 9 практических занятий, на каждом из которых команды (2–3 человека) выполняют последовательные этапы работы над сквозным проектом. Группа численностью ~25 человек формирует 8–11 команд, что позволяет обеспечить индивидуальную работу каждого участника и разнообразие проектных решений.

Все сформированные артефакты размещаются в репозитории команды.

*Распределение ролей в команде*

В каждой команде распределяются следующие роли (могут совмещаться):

| Роль                            | Обязанности                                                        |
|---------------------------------|--------------------------------------------------------------------|
| Руководитель проекта (PM)       | Координация работы, контроль сроков, коммуникация с преподавателем |
| Аналитик / Технический писатель | Сбор требований, ведение документации, оформление ТЗ               |
| Разработчик / Архитектор        | Проектирование прототипа, техническая реализация                   |

Система оценивания

- Каждое занятие — до 10 баллов (командная работа + индивидуальный вклад)
- Итоговая защита проекта — до 20 баллов
- Максимум за курс — 110 баллов
- Порог «зачтено» — 60 баллов, из них не менее 4 баллов за каждое занятие

*Сквозные индустриальные кейсы*

Команды выбирают один из кейсов на 1-м занятии и работают над ним весь семестр:

1. Рекомендательная система для онлайн-курсов
2. Система мониторинга цен на товары
3. Платформа для онлайн-опросов и голосований
4. Мобильное приложение для учёта личных финансов
5. Система управления задачами для студенческой команды

---

## **ЗАНЯТИЕ 1. Введение в проектную деятельность. Формирование команд**

### Цель занятия

Познакомиться с основами проектной деятельности, сформировать проектные команды, выбрать индустриальный кейс и первично очертить границы будущего проекта.

### Краткий теоретический конспект

Проект — это временное предприятие, направленное на создание уникального продукта, услуги или результата. Ключевые характеристики проекта:

- Ограниченность во времени (имеет начало и конец)
- Уникальность результата (отличие от операционной деятельности)
- Поэтапное развитие (жизненный цикл)
- Ресурсные ограничения (бюджет, люди, время)

Жизненный цикл ИТ-проекта включает 5 групп процессов (по PMBOK):

1. Инициация — определение идеи, целей, стейкхолдеров
2. Планирование — декомпозиция, оценка сроков, ресурсов, рисков
3. Исполнение — непосредственная реализация
4. Мониторинг и контроль — отслеживание отклонений
5. Закрытие — сдача результата, извлечение уроков

Отличие проекта от операционной деятельности:

| Проект | Операционная деятельность |
|--------|---------------------------|
|--------|---------------------------|

|                      |                           |
|----------------------|---------------------------|
| Временный            | Постоянная                |
| Уникальный результат | Повторяющиеся процессы    |
| Создаёт изменения    | Поддерживает стабильность |

Роли в проекте:

- Заказчик — инициатор, получатель результата
- Менеджер проекта (PM) — отвечает за достижение целей в рамках ограничений
- Команда — исполнители рабочих пакетов
- Стейкхолдеры — все заинтересованные лица

### Задание

Задание 1.1. Сформировать команды по 2–3 человека. Распределить роли внутри команды.

Задание 1.2. Выбрать один из 5 индустриальных кейсов (или предложить свой, согласовав с преподавателем).

Задание 1.3. Провести первичный мозговой штурм и сформулировать:

- Название проекта
- Краткое описание идеи (3–5 предложений)
- Целевую аудиторию
- Ключевую проблему, которую решает проект
- Ожидаемый конечный результат

Задание 1.4. Подготовить короткую (1 минута) самопрезентацию команды и проекта.

### Указания к выполнению

1. При формировании команды учитывайте разнообразие компетенций: желательно, чтобы в команде был участник с сильными техническими навыками и участник с навыками коммуникации/документирования.

2. При выборе кейса обсудите, какой из них наиболее интересен каждому участнику и соответствует вашим компетенциям.

3. Формулируя идею проекта, используйте принцип SMART: цель должна быть конкретной, измеримой, достижимой, релевантной и ограниченной по времени.

4. Результат занятия — заполненный шаблон «Визитка проекта» (1 страница), который сдаётся преподавателю.

5. При презентации говорите по очереди — каждый участник должен внести вклад.

 Результат занятия

- Заполненная «Визитка проекта» (1 стр.)
- Распределение ролей в команде
- Краткая устная презентация (1 минута)

---

## ЗАНЯТИЕ 2. Инициация проекта. Устав проекта

 Цель занятия

Научиться формализовать идею проекта в виде устава (Project Charter), определять стейкхолдеров и устанавливать первичные границы проекта.

 Краткий теоретический конспект

Устав проекта (Project Charter) — документ, официально иницирующий проект и наделяющий менеджера проекта полномочиями использовать ресурсы организации. Устав отвечает на вопросы: ЗАЧЕМ? ЧТО? КТО? В КАКИХ РАМКАХ?

Структура устава проекта:

1. Обоснование проекта — почему этот проект нужен
2. Цель проекта — конкретный измеримый результат
3. Описание продукта — что будет создано
4. Ключевые стейкхолдеры — кто заинтересован
5. Границы проекта (Scope) — что входит и что НЕ входит
6. Высокоуровневые риски — основные угрозы
7. Предварительный бюджет и сроки
8. Критерии успеха — как поймём, что проект удался

Анализ стейкхолдеров проводится по матрице Власть / Интерес:

- Высокая власть, высокий интерес — ключевые игроки, управление

близко

- Высокая власть, низкий интерес — удовлетворять, информировать
- Низкая власть, высокий интерес — информировать регулярно
- Низкая власть, низкий интерес — минимальный контроль

Техника SMART для целей:

- Specific — конкретная
- Measurable — измеримая
- Achievable — достижимая
- Relevant — релевантная
- Time-bound — ограниченная по времени

### Задание

Задание 2.1. Разработать устав проекта по следующему шаблону:

#### УСТАВ ПРОЕКТА

=====

1. Название проекта:
2. Обоснование (проблема/возможность):
3. Цель проекта (SMART):
4. Описание продукта (2-3 абзаца):
5. Границы проекта:
  - ВХОДИТ:
  - НЕ ВХОДИТ:
6. Ключевые стейкхолдеры (таблица):
 

| Стейкхолдер | Роль | Власть | Интерес | Стратегия |
|-------------|------|--------|---------|-----------|
|             |      |        |         |           |
|             |      |        |         |           |
|             |      |        |         |           |
7. Высокоуровневые риски (3-5 шт.):
8. Предварительные сроки:
9. Критерии успеха:

Задание 2.2. Построить карту стейкхолдеров (матрица Власть/Интерес) с указанием стратегии взаимодействия с каждой группой.

Задание 2.3. Сформулировать 3–5 высокоуровневых рисков проекта.

### Указания к выполнению

1. Устав должен быть лаконичным — не более 2 страниц. Избегайте воды, пишите по существу.

2. Цель проекта обязательно формулируйте по SMART. Пример плохой цели: «Сделать удобное приложение». Пример хорошей: «Разработать MVP мобильного приложения для учёта личных финансов с функциями категоризации расходов и построения отчётов за месяц, готового к использованию 100 тестовыми пользователями к 15 мая 2026 г.»

3. При определении границ проекта особенно важно указать, что НЕ ВХОДИТ в проект — это защитит команду от «расползания» содержания (scope creep).

4. Карту стейкхолдеров оформите в виде визуальной матрицы 2×2 с подписанными квадрантами.

5. Риски формулируйте по схеме: «Если <событие>, то <последствие>».



Результат занятия

- Устав проекта (до 2 стр.)
- Карта стейкхолдеров (визуальная матрица)
- Список высокоуровневых рисков

---

### ЗАНЯТИЕ 3. Сбор и формализация требований



Цель занятия

Освоить техники сбора требований, научиться формулировать функциональные и нефункциональные требования, создавать User Stories и приоритизировать бэклог продукта.



Краткий теоретический конспект

Требования к программному продукту — описание того, что должна делать система и каким характеристикам удовлетворять.

Виды требований:

- Бизнес-требования — высокоуровневые цели заказчика
- Пользовательские требования — что пользователи хотят делать с системой
- Функциональные требования — конкретные функции системы (что делает)

- Нефункциональные требования — характеристики качества (как работает): производительность, безопасность, надёжность, удобство использования

Техники сбора требований:

- Интервью — беседа с заказчиком/пользователем
- Анкетирование — массовый сбор данных
- Анализ документов — изучение существующей документации
- Мозговой штурм — генерация идей в группе
- Наблюдение — анализ работы пользователей
- Прототипирование — уточнение требований через макет

User Story — формат описания требования с точки зрения пользователя:

Как <роль пользователя>,  
я хочу <действие/функция>,  
чтобы <ценность/цель>.

Критерии INVEST для качественной User Story:

- Independent — независимая
- Negotiable — обсуждаемая
- Valuable — ценная
- Estimable — оцениваемая
- Small — маленькая
- Testable — тестируемая

Приоритизация по MoSCoW:

- Must have — обязательно (без этого продукт не работает)
- Should have — желательно (важно, но не критично)
- Could have — возможно (приятное дополнение)
- Won't have — не будет в этой версии (но может быть позже)

### Задание

Задание 3.1. Провести внутрикомандное «интервью»: один участник играет роль заказчика/пользователя, другие — аналитиков. Зафиксировать 10–15 ключевых потребностей.

Задание 3.2. Сформулировать требования в трёх категориях:

- 5–7 бизнес-требований
- 10–15 функциональных требований (в формате User Stories)
- 5–7 нефункциональных требований

Задание 3.3. Провести приоритизацию функциональных требований по методу MoSCoW.

Задание 3.4. Сформировать первоначальный бэклог продукта — упорядоченный список User Stories.

#### Указания к выполнению

1. При интервью используйте открытые вопросы: «Что вы хотите получить?», «Какие задачи решаете сейчас?», «Что вас не устраивает в существующих решениях?»

2. Функциональные требования формулируйте однозначно. Избегайте слов «удобно», «быстро», «красиво» — это нефункциональные характеристики.

3. Нефункциональные требования должны быть измеримы. Плохо: «система должна работать быстро». Хорошо: «время отклика системы при стандартных операциях — не более 2 секунд при 100 одновременных пользователях».

4. В MoSCoW-категорию Must have отнесите не более 40–50% требований — это ваш MVP.

5. Бэклог продукта оформите в виде таблицы с колонками: ID | User Story | Приоритет (MoSCoW) | Оценка сложности (1–5) | Статус.

#### Результат занятия

- Протокол интервью (1 стр.)
- Список бизнес-, функциональных и нефункциональных требований
- Приоритизированный бэклог продукта (таблица)

---

## ЗАНЯТИЕ 4. Декомпозиция проекта. Построение WBS

### Цель занятия



Освоить метод иерархической декомпозиции работ (WBS), научиться разбивать проект на управляемые рабочие пакеты и оценивать трудозатраты.

### Краткий теоретический конспект

WBS (Work Breakdown Structure) — иерархическая декомпозиция общей\_суммы\_ проекта на более мелкие, управляемые компоненты.

Принципы построения WBS:

1. Правило 100% — WBS должен включать 100% работы по проекту, ни больше, ни меньше
2. Взаимное исключение — элементы не должны пересекаться
3. Управляемость — нижний уровень (рабочий пакет) должен быть достаточно мал для оценки и контроля
4. Ориентация на результат — элементы WBS описывают результаты, а не действия

Уровни WBS:

- Уровень 0 — проект целиком
- Уровень 1 — основные фазы/разделы
- Уровень 2 — компоненты/подфазы
- Уровень 3 — рабочие пакеты (минимальный уровень декомпозиции)

Методы оценки трудозатрат:

- Экспертная оценка — мнение опытных специалистов
- Метод Дельфи — многократный опрос экспертов с анонимной обратной связью
- Оценка по аналогии — на основе похожих прошлых проектов
- Оценка по трём точкам (PERT):

$$E = (O + 4M + P) / 6$$

где O — оптимистичная, M — наиболее вероятная, P — пессимистичная оценка

Правило 8/80: рабочий пакет должен выполняться не менее 8 и не более 80 часов.

### Задание

Задание 4.1. Построить WBS проекта минимум в 3 уровня декомпозиции. Использовать графическое представление (древовидная схема).

Задание 4.2. Для каждого рабочего пакета нижнего уровня оценить трудозатраты методом оценки по трём точкам (PERT).

Задание 4.3. Составить словарь WBS — краткое описание каждого элемента.

Задание 4.4. Проверить WBS на соответствие правилу 100%: все ли работы из устава проекта учтены?

#### Указания к выполнению

1. Начните с определения основных фаз проекта (уровень 1). Для ИТ-проекта типичные фазы: Анализ, Проектирование, Разработка, Тестирование, Внедрение, Закрытие.

2. Декомпозируйте до тех пор, пока рабочие пакеты не станут оцениваемыми и управляемыми. Обычно 3–4 уровня достаточно.

3. Рабочие пакеты формулируйте как результат (существительное), а не как действие (глагол). Плохо: «Разрабатывать модуль авторизации». Хорошо: «Модуль авторизации пользователей».

4. При оценке по PERT используйте реальные данные: вспомните похожие задачи из курса «Основы программирования».

5. Для построения WBS используйте инструменты: draw.io, XMind, Miro или бумагу + маркеры.

6. Словарь WBS — это таблица: ID | Название элемента | Описание | Ответственный | Оценка трудозатрат.

#### Результат занятия

- Графическая схема WBS (3 уровня)
- Таблица оценок трудозатрат (с расчётом PERT)
- Словарь WBS

---

## ЗАНЯТИЕ 5. Календарное планирование. Диаграмма Ганта

### Цель занятия

Научиться строить календарный план проекта, определять критический путь и визуализировать расписание в виде диаграммы Ганта.

### Краткий теоретический конспект

Календарное планирование — определение сроков начала и окончания работ, их взаимосвязей и распределения во времени.

Диаграмма Ганта — горизонтальная линейная диаграмма, показывающая расписание проекта:

- По оси X — время
- По оси Y — задачи
- Горизонтальные полосы — длительность задач

Типы взаимосвязей задач:

- FS (Finish-to-Start) — завершение-начало (самая частая)
- SS (Start-to-Start) — начало-начало
- FF (Finish-to-Finish) — завершение-завершение
- SF (Start-to-Finish) — начало-завершение (редко)

Критический путь (Critical Path) — самая длинная последовательность задач, определяющая минимальную длительность проекта. Задачи на критическом пути не имеют запаса — их задержка задержит весь проект.

Метод критического пути (CPM):

1. Построить сетевой график
2. Определить ранние сроки начала/окончания (прямой проход)
3. Определить поздние сроки (обратный проход)
4. Рассчитать резервы: Резерв = Поздний срок – Ранний срок
5. Задачи с нулевым резервом — на критическом пути

Инструменты построения:

- GanttProject (бесплатный)
- MS Project
- LibreOffice / Excel (вручную)
- Онлайн-сервисы: TeamGantt, ClickUp

### Задание

Задание 5.1. На основе WBS и оценок трудозатрат определить последовательности задач и их взаимосвязи.

Задание 5.2. Построить диаграмму Ганта в GanttProject (или аналоге) с учётом:

- Длительности задач
- Взаимосвязей между задачами
- Выходных дней
- Вех (milestones) — ключевых контрольных точек

Задание 5.3. Определить критический путь проекта. Выделить его на диаграмме.

Задание 5.4. Рассчитать общий срок проекта и резервы для некритических задач.

Задание 5.5. Сформулировать 4–6 ключевых вех проекта с датами.

 Указания к выполнению

1. Начните с определения зависимостей: какие задачи не могут начаться, пока не завершены другие.

2. Используйте FS-зависимости как базовые, остальные — при необходимости.

3. Вехи (milestones) — это задачи с нулевой длительностью, обозначающие важные события: «ТЗ согласовано», «Прототип готов», «Защита проекта».

4. При построении диаграммы учитывайте, что один участник может выполнять только одну задачу в единицу времени.

5. Критический путь выделите красным цветом.

6. Если срок проекта превышает разумные рамки — ищите возможности параллельного выполнения задач.

7. Сохраните файл проекта в формате .gan (GanttProject) и экспортируйте в PDF для защиты.

 Результат занятия

- Диаграмма Ганта (файл проекта + PDF)

- Выделенный критический путь
  - Список вех проекта (таблица)
  - Расчёт резервов
- 

## ЗАНЯТИЕ 6. Управление ресурсами. Матрица RACI

### Цель занятия

Научиться планировать ресурсы проекта, распределять ответственность с помощью матрицы RACI и балансировать загрузку команды.

### Краткий теоретический конспект

Ресурсы проекта — всё, что необходимо для выполнения работ:

- Трудовые ресурсы — участники команды, их компетенции
- Временные ресурсы — доступное время каждого участника
- Материальные ресурсы — оборудование, ПО, помещения
- Финансовые ресурсы — бюджет

Матрица RACI — инструмент распределения ответственности:

- R (Responsible) — исполнитель, тот, кто делает работу
- A (Accountable) — ответственный, тот, кто принимает работу (один на задачу!)
- C (Consulted) — консультируемый, эксперт, чьё мнение учитывают
- I (Informed) — информируемый, тот, кого держат в курсе

Правила матрицы RACI:

- На каждую задачу — ровно одна А
- На каждую задачу — хотя бы одна R
- Минимизировать количество R (избегайте «все отвечают»)
- Осторожно с С — не превращайте в формальность

Балансировка загрузки:

- Выравнивание ресурсов — сдвиг задач в пределах резервов для устранения перегрузки
- Сглаживание ресурсов — может увеличить срок проекта, но снижает пики загрузки

Планирование загрузки:

- 100% загрузка = 8 часов в день (норма)
- Оптимальная загрузка — 80% (остальное — на непредвиденное)
- Перегрузка >100% ведёт к выгоранию и ошибкам

#### Задание

Задание 6.1. Составить матрицу RACI для всех рабочих пакетов WBS. Участники команды — строки, рабочие пакеты — столбцы.

Задание 6.2. Построить диаграмму загрузки каждого участника команды на период проекта.

Задание 6.3. Выявить перегруженных и недогруженных участников. Предложить меры по балансировке.

Задание 6.4. Описать требуемые материальные и финансовые ресурсы проекта.

Задание 6.5. Составить предварительный бюджет проекта (таблица затрат).

#### Указания к выполнению

1. Матрица RACI — это таблица. Строки — рабочие пакеты WBS (10–15 шт.), столбцы — участники команды (2–3 человека) + преподаватель/заказчик как консультант.

2. Проверяйте матрицу на корректность: в каждом столбце ровно одна А, хотя бы одна R.

3. Если у одного участника слишком много R — это риск перегрузки. Перераспределите задачи.

4. Диаграмму загрузки постройте в Excel: по оси X — недели проекта, по оси Y — часы загрузки.

5. Учитывайте, что студенты совмещают проект с другими дисциплинами. Реальная доступность — 6–10 часов в неделю на проект.

6. Бюджет проекта — учебный, но реалистичный. Включите: трудозатраты команды (в условных единицах), лицензии ПО, оборудование, возможные внешние услуги.

#### Результат занятия

- Матрица RACI (таблица)
- Диаграмма загрузки команды
- План балансировки ресурсов
- Перечень ресурсов и бюджет проекта

---

## ЗАНЯТИЕ 7. Управление рисками

### Цель занятия

Освоить процесс идентификации, анализа и планирования реагирования на проектные риски.

### Краткий теоретический конспект

Риск проекта — неопределённое событие или условие, которое в случае наступления положительно или отрицательно влияет на цели проекта.

Процесс управления рисками:

1. Идентификация — выявление возможных рисков
2. Качественный анализ — оценка вероятности и влияния
3. Количественный анализ — числовая оценка воздействия  
(опционально)
4. Планирование реагирования — разработка стратегий
5. Мониторинг — отслеживание рисков и триггеров

Матрица вероятности и влияния:

|               | Очень низкое | Низкое | Среднее | Высокое | Очень высокое |
|---------------|--------------|--------|---------|---------|---------------|
| Очень высокая | 2            | 4      | 6       | 8       | 10            |
| Высокая       | 2            | 4      | 6       | 8       | 10            |
| Средняя       | 1            | 3      | 5       | 7       | 9             |
| Низкая        | 1            | 2      | 4       | 6       | 8             |
| Очень низкая  | 1            | 2      | 3       | 4       | 6             |

Стратегии реагирования на негативные риски:

- Избегание — устранить причину риска
- Передача — передать третьей стороне (страхование, аутсорсинг)
- Снижение — уменьшить вероятность или влияние

- Принятие — осознанно принять (активное — с резервом, пассивное — без действий)

Категории рисков (Risk Breakdown Structure):

- Технические — технологии, архитектура, интеграция
- Внешние — заказчик, регуляторы, рынок
- Организационные — приоритеты, ресурсы
- Управленческие — планирование, оценка, коммуникации

Техники идентификации рисков:

- Мозговой штурм
- Анализ допущений
- Чек-листы (на основе прошлых проектов)
- SWOT-анализ
- Анализ корневых причин

#### Задание

Задание 7.1. Провести мозговой штурм по идентификации рисков проекта. Выявить минимум 10–15 рисков.

Задание 7.2. Классифицировать риски по категориям (технические, внешние, организационные, управленческие).

Задание 7.3. Для каждого риска оценить вероятность (1–5) и влияние (1–5). Рассчитать приоритет.

Задание 7.4. Построить матрицу рисков (визуально — «тепловая карта»).

Задание 7.5. Для топ-5 рисков разработать планы реагирования: стратегия, конкретные действия, ответственный, триггер срабатывания.

Задание 7.6. Определить резерв на непредвиденные работы (обычно 10–20% от срока/бюджета).

#### Указания к выполнению

1. При мозговом штурме используйте технику «Что может пойти не так?» по каждой фазе проекта.



2. Не ограничивайтесь очевидными рисками. Подумайте о рисках команды (болезнь участника, конфликты), рисках требований (изменение от заказчика), технических рисках (неподходящая технология).

3. Матрицу рисков визуализируйте в виде цветной сетки 5×5, где цветом показана зона: зелёная (низкий риск), жёлтая (средний), красная (высокий).

4. План реагирования на риск — это таблица:

|           |          |               |         |           |
|-----------|----------|---------------|---------|-----------|
| ID риска  | Описание | Вероятность   | Влияние | Приоритет |
| Стратегия | Действия | Ответственный | Триггер | Резерв    |

5. Триггер — это конкретное событие, при наступлении которого план реагирования активируется.

6. Для учебных проектов типичные риски: срыв сроков из-за других дисциплин, уход участника из команды, изменение требований, технические сложности.

 Результат занятия

- Реестр рисков (таблица с 10–15 рисками)
- Матрица рисков (визуальная «тепловая карта»)
- Планы реагирования на топ-5 рисков
- Расчёт резерва проекта

---

## ЗАНЯТИЕ 8. Проектная документация. Техническое задание

 Цель занятия

Освоить разработку технического задания (ТЗ) в соответствии с ГОСТ 34, научиться структурировать проектную документацию.

 Краткий теоретический конспект

Техническое задание (ТЗ) — основной документ, определяющий требования к создаваемой системе и порядок приёмки.

Стандарты технической документации:

- ГОСТ 34.602-89 — ТЗ на создание автоматизированной системы
- ГОСТ 19.101-77 — Виды программ и программных документов

- ГОСТ 19.201-78 — Техническое задание (для программных средств)
- ГОСТ 19.401-78 — Описание программы

Структура ТЗ по ГОСТ 34.602-89:

1. Общие сведения
2. Назначение и цели создания системы
3. Характеристика объектов автоматизации
4. Требования к системе
  - о Требования к системе в целом
  - о Требования к функциям (подсистемам)
  - о Требования к видам обеспечения (техническому, математическому, информационному, программному, организационному)
5. Состав и содержание работ по созданию системы
6. Порядок контроля и приёмки системы
7. Требования к составу работ по подготовке объекта автоматизации
8. Требования к документированию
9. Источники разработки

Сопроводительная документация:

- Руководство пользователя — как работать с системой
- Руководство администратора — как настраивать и обслуживать
- Руководство программиста — как развивать и модифицировать
- Описание архитектуры — структура и принципы системы
- Описание API — интерфейсы взаимодействия

Современные подходы к документации:

- Markdown — для описания в репозитории
- Swagger/OpenAPI — для описания API
- UML-диаграммы — для визуализации архитектуры
- ADR (Architecture Decision Records) — для фиксации архитектурных решений

 Задание

Задание 8.1. Разработать техническое задание на проект по структуре ГОСТ 34.602-89 (адаптированной для учебного проекта).

Задание 8.2. Описать функциональные требования в виде таблицы с указанием ID, описания, приоритета, критериев приёмки.

Задание 8.3. Разработать описание архитектуры проекта:

- Общая схема (компоненты, связи)
- стек технологий
- Схема базы данных (если применимо)
- UML-диаграмма Use Case

Задание 8.4. Подготовить краткое руководство пользователя (1–2 стр.) с описанием основных сценариев работы.

Задание 8.5. Оформить всю документацию в едином репозитории Git с файлом README.md.

#### Указания к выполнению

1. ТЗ — это официальный документ. Соблюдайте формальный стиль, нумерацию разделов, точность формулировок.

2. Адаптируйте ГОСТ под учебный проект: не все разделы обязательны в полном объёме, но структура должна соблюдаться.

3. Функциональные требования оформите таблицей:

| ID | Название | Описание | Приоритет | Критерий приёмки |
|----|----------|----------|-----------|------------------|
|----|----------|----------|-----------|------------------|

4. Архитектуру опишите с помощью draw.io или PlantUML. Экспортируйте в PNG и вставьте в документ.

5. UML-диаграмму Use Case постройте для 3–5 основных сценариев использования.

6. Руководство пользователя пишите простым языком, с примерами и скриншотами (если есть прототип).

7. Структура репозитория:

```
/project-name  
/docs
```

```
/technical-specification.md
/architecture.md
/user-manual.md
/src
/tests
/README.md
```

8. README.md должен содержать: название, краткое описание, стек технологий, инструкции по запуску, ссылку на документацию.

#### Результат занятия

- Техническое задание (10–15 стр.)
- Описание архитектуры (схема + текст)
- UML-диаграмма Use Case
- Руководство пользователя
- Структурированный репозиторий проекта

---

## ЗАНЯТИЕ 9. Подготовка и защита проекта (питч)

### Цель занятия

Подготовить презентационные материалы, отработать навыки публичного выступления, провести итоговую защиту проекта перед комиссией.

### Краткий теоретический конспект

Питч (pitch) — краткое убедительное представление проекта перед аудиторией с целью получить одобрение, финансирование или поддержку.

Структура эффективного питча (10 слайдов по Guy Kawasaki):

1. Название — логотип, слоган, суть в одном предложении
2. Проблема — какую боль решаете
3. Решение — как именно решаете
4. Демонстрация — работающий прототип или скриншоты
5. Бизнес-модель — как будет работать (для учебного — ценность для пользователя)
6. Рынок / Аудитория — кто будет пользоваться

7. Команда — кто делает, почему именно вы
8. Конкуренты — чем отличаетесь
9. Прогноз / План — что дальше
10. Призыв к действию — что хотите от аудитории

Правила создания слайдов:

- 10/20/30 (Guy Kawasaki): 10 слайдов, 20 минут, шрифт 30+
- Один слайд — одна мысль
- Минимум текста, максимум визуализации
- Читаемость с задних рядов
- Единый стиль оформления
- Избегать «стены текста»

Техники публичного выступления:

- Начните с крючка — вопрос, статистика, история
- Правило трёх — три ключевые идеи
- Контакт с аудиторией — взгляд, паузы, вопросы
- Репетиция — минимум 3 раза вслух
- Управление временем — уложиться в регламент
- Работа с вопросами — не бояться, отвечать честно

Формат защиты:

- Презентация — 5–7 минут
- Демонстрация прототипа — 2–3 минуты
- Вопросы комиссии — 3–5 минут
- Итого: ~10–15 минут на команду

#### Задание

Задание 9.1. Разработать pitch deck из 10 слайдов по структуре выше.

Задание 9.2. Подготовить демонстрацию прототипа (рабочего или макета).

Задание 9.3. Распределить роли при защите: кто какие слайды представляет, кто демонстрирует прототип, кто отвечает на вопросы.

Задание 9.4. Провести репетицию защиты с таймингом. Получить обратную связь от другой команды.

Задание 9.5. Подготовить ответы на типичные вопросы комиссии:

- Почему выбрали эту технологию?
- Какие риски видите и как с ними работаете?
- Что бы сделали иначе, если начать сначала?
- Как распределяли нагрузку в команде?
- Какие метрики успеха использовали?

Задание 9.6. Провести итоговую защиту проекта перед комиссией.

 Указания к выполнению

1. Pitch deck оформите в едином стиле. Используйте Canva, PowerPoint, LibreOffice Impress или Google Slides.
2. На слайдах — минимум текста. Ключевые цифры, схемы, скриншоты. Всё остальное — в устном выступлении.
3. Демонстрация прототипа — ключевой момент. Подготовьте сценарий демонстрации: что показываете, в каком порядке, какие функции акцентируете.
4. Если прототип не работает — покажите макет, схему, архитектуру. Главное — продемонстрировать проработанность решения.
5. Репетиция обязательна! Засеките время. Убедитесь, что все участники выступают примерно поровну.
6. При защите говорите уверенно, смотрите на аудиторию, не читайте со слайдов.
7. На вопросы отвечайте честно. Если не знаете — скажите об этом и предложите, как будете разбираться.
8. После защиты проведите ретроспективу команды: что получилось, что нет, какие уроки извлекли.

 Результат занятия

- Pitch deck (10 слайдов)
- Демонстрация прототипа
- Итоговая защита проекта
- Ретроспектива команды (1 стр.)